Proyecto 1
Fundamentos de Bases de Datos

Tomás Luengas Gaitán - 202422456
Emilio Molina Mahecha - 202422272
Sara Hernandez - 202412681

1. Introducción

En el presente documento se describe el diseño del proyecto 1 de fundmantos de bases de datos. Para este se desarollo un sistema de gestión de reservas de la cadena de hotelería Dann-Alpes. El objetivo principal de este sistema es poder soportar de manera estructurada las operaciones relacionales entre las entidades del hotel, las cuales incluyen las habitaciones, servicios y reservas realizadas por los clientes.
A partir de los requerimientos funcionales del negocio, se realizó el ejercicio de identificar las entidades, sus atributos/caracteristicas y las relaciones y restricciones entre estas. Luego se hizo un modelo conceptual (Entidad//Relación). Posteriormente, se realizó un modelo relacional y se revisó que contara con todas las reglas de normalización para hacer las relaciones lo más eficientes posibles. Por ultimó, para asegurar que se realizaron correctamente las tablas, se creó un escenario de pruebas que se divide en las pruebas de PK, FK, CHECH y caso de negocio.

2. Modelo Conceptual

2.1 Entidades modelo

- Usuarios
- Clientes
- Administradores
- Reservas
- Hoteles
- Ciudad
- Tipo de Habitaciones
- Servicio
- Servicio Costo Variable
- Servicio Con Costo Fijo

2.2 Requerimientos funcionales:

2.2.1 Requisitos funcionales Administrador:

RF1 – Registrar ciudad: Registrar una nueva ciudad en la que exista un hotel de Dann-Alpes.

RF2 – Registrar hotel: Registrar un hotel en una ciudad previamente registrada. Una misma ciudad puede tener múltiples hoteles.

RF2.5-Registrar servicio: Registrar un servicio indicando si se aplica una sola vez o por noche.

RF3 – Registrar habitaciones para un hotel: Registrar un tipo de habitación para un hotel, incluyendo nombre, capacidad, dotación, tipo de cama, costos por noche en temporada alta y baja, número de habitaciones disponibles y servicios adicionales asociados. Debe considerarse que algunos servicios aplican por reserva (p. Ej.: check-in temprano, valet parking) y otros por persona (ej: desayuno, cuyo costo se multiplica por número de huéspedes).

RF4- Editar habitaciones: Editar la información de un tipo de habitación según las necesidades del negocio: nombre, costo, tipo y cantidad de camas, y amenidades incluidas. Esta edición aplica por hotel de manera independiente.

2.2.2 Requisitos funcionales Cliente:

RF5 – Creación de cuenta de cliente: Registrar la información de un cliente (identificación, nombre, correo electrónico y teléfono). Esta operación puede ser realizada tanto por el cliente como por el administrador.

RF6– Gestión de cuenta de cliente: El cliente puede modificar sus datos personales de ser necesario.

RF7 – Realizar reserva de habitación: Registrar una reserva realizada por un cliente. La reserva debe incluir: cliente, tipo de habitación, fechas de entrada y salida, número de personas adultas y menores de 3 años, servicios adicionales seleccionados y el precio total. La reserva solo puede efectuarse si hay disponibilidad.

RF7 – Gestionar la reserva por parte del cliente: El cliente podrá consultar los detalles de su reserva, modificar las fechas o cancelarla con al menos tres días de anticipación al inicio de la estadía.

2.2.3 Requisitos funcionales Administrador (Consulta)

RFC1 - Consultar habitaciones disponibles para un intervalo de fechas en un hotel particular con ciertos criterios. Cualquier usuario, incluso sin cuenta, puede consultar las habitaciones disponibles en un hotel para un intervalo de fechas.

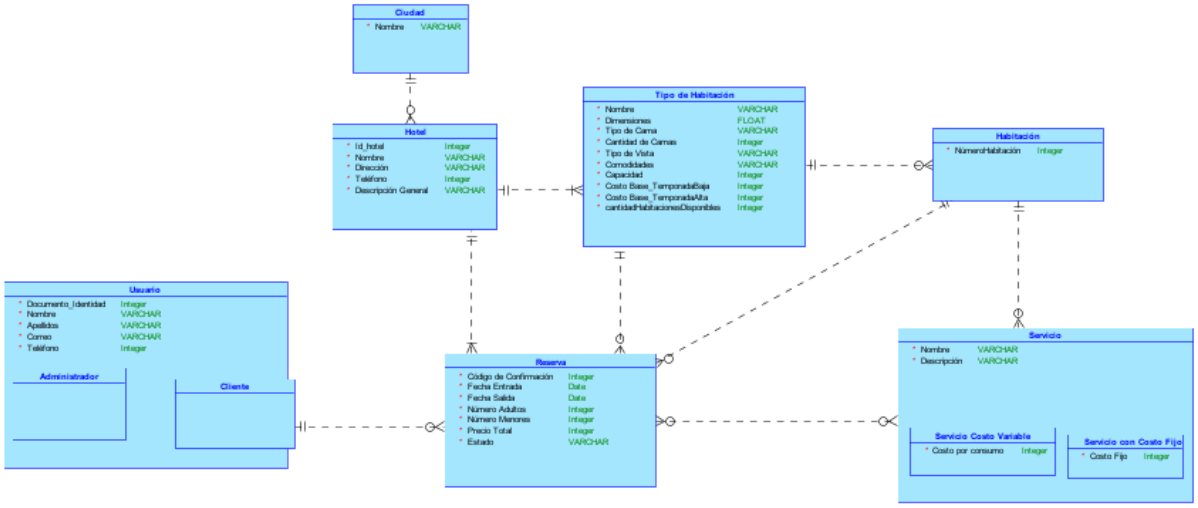
RFC2 – Consultar reservas a través del tiempo: Permite visualizar la evolución de las reservas en un intervalo de tiempo determinado para identificar las temporadas con mayor flujo de visitantes. Se debe poder filtrar por hotel y por ciudad.

RFC3 – Consultar la cantidad de huéspedes por fechas y ciudad: Muestra el número total de huéspedes en fechas específicas para una ciudad determinada y para todos los hoteles en dicha ciudad, considerando que una reserva puede incluir más de un huésped. Se debe poder filtrar por hoteles para ver cuántos huéspedes hay en un hotel específico.

RFC4 – Consultar el ingreso total por hotel por rango de fechas: Permite visualizar los ingresos generados por cada hotel durante un año determinado, obteniendo un agregado por ciudad y por hotel.

RFC5 – Consultar los servicios y tipos de habitación más populares por sede por rango de fechas: Permite identificar cuáles son los servicios y tipos de habitación más utilizados, con el fin de determinar las preferencias de los clientes y orientar futuras estrategias de negocio. (Servicios más populares y configuraciones, según las reservas realizadas).

2.2.4 Presentacion del Modelo conceptual



2.2.4 Logica del modelo (relaciones):

1. Usuario, administrador y cliente pueden tener de 0 a muchas reservas.
2. Una reserva siempre debe de tener un usuario, administrador o cliente asociado.
3. Ciudad puede tener de 0 a muchos hoteles.
4. Un hotel siempre debe de tener un hotel asociado.
5. Un hotel tiene que tener 1 a muchas reservas.

6. Una reserva tiene que tener un hotel asociado.
7. Una reserva tiene que tener un tipo de habitacion asociada.
8. Un tipo de reserva puede tener de 0 a muchas reservas.
9. Un hotel tiene que tener 1 a muchos tipos de habitaciones
10. Un tipo de habitación tiene que tener un hotel asociado.
11. Un tipo de reserva puede tener de 0 a muchas habitaciones.
12. Una habitación tiene que tener un tipo de habitacion.
13. Una reserva puede tener de 0 a muchos servicios.
14. Un servicio puede tener de 0 a muchas reservas.
15. Un servicio tiene que tener una habitacion.
16. Una habitación puede tener de 0 a muchos servicios.
17. Una reserva tiene que tener una habitacion asociada.
18. Una habitacion puede tener de 0 a muchas reservas.

3. Modelo Relacional

A partir del modelo conceptual previamente presentado, se realizó la siguiente transformación al modelo relacional visto en clase:

3.1 Ciudad

- Tiene como llave primaria “id_ciudad”.
- Tiene el atributo “nombre” y evita que estas se repitan.

Ciudad	
PK, NOT NULL	Unique, NOT NULL
id_ciudad	nombre

3.2 Hotel

- Se establece “id_hotel” como la llave primaria.

- “id_ciudad” es la llave foránea (Establece relación con ciudad).

Hotel					
PK, NOT NULL	FK, NOT NULL	NOT NULL	NOT NULL		
id_hotel	id_ciudad	nombre	direccion	telefono	descripcion_general

3.3 Usuario

- Se establece “documento” como llave primaria.
- Atributo “correo” es único en todo el sistema.

Usuario				
PK, NOT NULL	NOT NULL	NOT NULL	NOT NULL, UNIQUE	
documento	nombres	apellidos	correo	telefono

3.4 Cliente

- “documento” como llave primaria y foránea para relacionar con usuario.

Cliente
PF, FK, NOT NULL
documento

3.5 Administrador

- “documento” como llave primaria y foránea para relacionar con usuario.

Administrador
PK, FK, NOT NULL
documento

3.6 Tipo_habitación

- “id_tipo” se establece como llave primaria.
- “id_hotel” llave foreánea que indica a que hotel pertenece.

Tipo_Habitacion								
PK, NOT NULL	FK, NOT NULL	NOT NULL		NOT NULL, CHECK(capacidad > 0)		NOT NULL, CHECK(costo_alta >= 0)	NOT NULL, CHECK(costo_baja >= 0)	NOT NULL, CKECK(cantidad_disponible >= 0)
id_tipo	id_hotel	nombre	dimensiones_m2	capacidad	tipo_vista	costo_alta	costo_baja	cantidad_disponible

3.7 Habitación

- “id_tipo” indica el tipo de habitación a la que pertenece.
- Llave primaria compuesta utilizando “id_hotel” y “numero_habitacion”.

Habitacion		
PK. FK, NOT NULL	PK, NOT NULL	FK, NOT NULL
id_hotel	numero_habitacion	id_tipo

3.8 Servicio

- “id_servicio” se establece como llave primaria.
- Atributo “nombre” es único.
- “tipo_costo” define si el costo es fijo o por consumo.
- “modo_cobro” indica si se cobra por reserva, por noche o por persona.

Servicio					
PK, NOT NULL	NOT NULL, UNIQUE		CHECK(tipo_costo IN "FIJO", "CONSUMO")	CHECK(modo_cobro in "POR_RESERVA","POR_NOCHE", "POR_PERSONA")	CHECK(precio_base >= 0)
id_servicio	nombre	descripcion	tipo_costo	modo_cobro	precio_cobro

3.9 Tipo_habitación_servicio

- Llave primaria compuesta por atributos “id_tipo” y “id_servicio”.

Tipo_habita_servicio			
PK,FK, NOT NULL	PK, FK, NOT NULL	CHECK(sobreCosto >= 0)	CHECK(habilidado IN (0, 1))
id_Tipo	id_servicio	sobreCosto	habilitado

3.10 Reserva

- “codigo_reserva” es la llave primaria.
- “documento_cliente” es la llave foránea que establece la relación con Cliente.

Reserva									
PK, NOT NULL	FK, NOT NULL	FK, NOT NULL	FK, NOT NUL	NOT NULL	NOT NULL	CHECK(num_adultos >= 1)	CHECK(num_menores >= 0)	CHECK(estado IN "ACTIVA", "CANCELADA", "FINALIZADA")	CHECK(precio_total >= 0)
codigo_reserva	documento_cliente	id_hotel	id_tipo	fecha_entrada	fecha_salida	num_adultos	num_menores	estado	precio_total

3.11 Reserva_servicio

- Llave primaria compuesta por “codigo_reserva” y “id_servicio”.
- Permite almacenar precio unitario (precio del servicio en el momento de la reserva), subtotal (total pagado por ese servicio en la reserva) y cantidad.

Reserva_Servicio				
PK. FK, NOT NULL	PK, FK, NOT NULL	CHECK(cantidad >= 1)	CHECK(precio_unitario >= 0)	CHECK(subtotal >= 0)
codigo_reserva	id_servicio	cantidad	precio_unitario	subtotal

4. Normalización

- A continuación se presentarán las formas de normalización, utilizadas para tener una buena estructura libre de redundancia:

4.1 Primera Forma Normal

- En el modelo relacional propuesto se puede evidenciar el correcto cumplimiento de la Primera Forma Normal, ya que cada uno de los atributos o casillas contiene un único valor (no multivariado), evitando la presencia de conjuntos o listas dentro de un mismo campo. La aplicación de esta norma en el proyecto se evidencia principalmente al crear las tablas intermedias “Tipo_habitacion_servicio” y “Reserva_servicio”, eliminando de esta manera cualquier forma de atributo multivaluado. Asimismo, la separación entre “Tipo_habitacion” y “Habitacion” contribuye a mantener la atomicidad de los datos, evitando la repetición innecesaria de información estructural en una misma relación.

4.2 Segunda Forma Normal

- En cuanto a la Segunda Forma Normal, el modelo cumple con esta debido a que en las tablas que poseen claves primarias compuestas, como “Habitacion” y “Reserva_servicio”, los atributos no dependen únicamente de una parte de la llave primaria, sino de la clave completa. Esto permite garantizar que no existan dependencias parciales entre los atributos. Por ejemplo, en la tabla “Reserva_servicio”, los atributos como cantidad, precio_unitario y subtotal dependen tanto del código de la reserva como del servicio asociado, y no de uno solo de ellos. De esta manera se evita redundancia y se asegura coherencia estructural.

4.3 Tercera Forma Normal

- El modelo también cumple con la Tercera Forma Normal debido a que no existen dependencias transitivas entre atributos no clave. Es decir, ningún atributo que no forme parte de la llave primaria depende de otro atributo que tampoco sea llave. Por ejemplo, en la entidad “Hotel”, los atributos como direccion, telefono y descripcion_general dependen únicamente de la llave primaria id_hotel y no presentan relación entre sí. Esto permite eliminar redundancias indirectas y garantiza que cada dato se almacene en el lugar que le corresponde dentro del modelo.

4.4 Cuarta Forma Normal

- Por último, este modelo también satisface la Cuarta Forma Normal debido a que no presenta dependencias multivaluadas independientes dentro de una misma relación. Las asociaciones múltiples fueron correctamente descompuestas en tablas intermedias, como “Tipo_habitacion_servicio” y “Reserva_servicio”, evitando que una sola tabla almacene múltiples conjuntos de datos independientes. Esto asegura que cada relación contenga información relacionada directamente con su clave primaria, eliminando posibles redundancias estructurales y manteniendo la consistencia del modelo en escenarios donde una entidad pueda asociarse con múltiples elementos de forma independiente.

5. Escenarios de prueba

Pruebas de PK

PK-1 Ciudad

- Tabla: Ciudad
- Acción: Insertar
- Datos caso exitoso: id_ciudad = 1, nombre = "Bogotá"
- Resultado esperado: Se inserta (PK nueva).
- Datos caso fallado: id_ciudad=1, nombre="Medellín"
- Resultado esperado: Falla por PK duplicada, Bogotá ya tiene como identificador único en número “1”

PK-2 Hotel

- Tabla: Hotel
- Acción: Insertar

- Dato caso exitoso: id_ hotel = 10, id_ciudad = 1, nombre = "4 seasons", dirección="Calle X", telefono="123", descripción general="Hotel con comida incluida y salas de estudio"

- Esperado: Inserta el hotel 4 semanas de forma correcta.

- Dato caso fallido: id_ hotel= 10

- Esperado: Falla por PK duplicada.

PK-3 Servicio

- Tabla: Servicio

- Acción: Insertar

- Dato caso exitoso: id_servicio=5, nombre="Almuerzo", descripcion="Frijoles con arroz", tipo_costo="$$", modo_cobro="efectivo", precio_cobro=30000

- Esperado: Inserta correctamente el almuerzo.

- Falla (PK): volver a insertar id_servicio=5

- Esperado: Falla por PK duplicada.

- Datos caso fallado : id_servicio=6, nombre="Almuerzo" (mismo nombre)

- Esperado: Falla por nombre único.

PK-4 Reserva

- Tabla: Reserva

- Acción: Insertar

- : codigo_reserva="Reserva1", documento_cliente="1111111111", id_hotel=10, id_tipo=1, fecha_entrada=2026-03-10, fecha_salida=2026-03-12, num_adultos=2, num_menores=0, estado="Activa", precio_total=400000

- Esperado: Inserta la reserva correctamente

- Dato caso fallado Falla: codigo_reserva="R001"

- Esperado: Falla por PK duplicada.

PK-5 Habitacion

- Tabla: Habitacion
- Acción: Insertar
- OK: id_hotel=10, numero_habitacion=101, id_tipo=1
- Esperado: Inserta.
- Dato caso fallido: id_hotel=10, numero_habitacion=102
- Esperado: Falla por PK duplicada

PK-6 Reserva_Servicio

- Tabla: Reserva_Servicio
- Acción: Insertar
- Dato caso exitoso: codigo_reserva="Reserva1", id_servicio=5, cantidad=2, precio_unitario=30000, subtotal=60000
- Esperado: Inserta la reserva con los precios de forma correcta
- Dato caso fallado: Codigo_reserva = “Reserva1”, id_servicio = 6
- Esperado: Falla por PK compuesta duplicada aunque los datos sean distintos

PK-7 Tipo_habita_servicio

- Tabla: Tipo_habita_servicio
- Acción: Insertar
- Dato caso exitoso: id_tipo=1, id_servicio=5, sobreCosto=0, habilitado=1
- Esperado: Inserta el servicio con el sobrecosto(si es que tiene)
- Dato caso fallado: id_tipo=1, sobreCosto=0, habilitado = 1
- Esperado: Falla por PK compuesta duplicada.

2) Pruebas de FK

FK-1 Hotel a Ciudad

- Acción: Insertar Hotel
- Dato caso exitoso: crear primero Ciudad, id_ciudad=1. Luego Hotel con id_ciudad=1
- Esperado: Inserta correctamente el hotel que tiene como id “1”
- dato caso fallado: Hotel con id_ciudad=100000000000000000000
- Esperado: Falla por FK, no existe una ciudad 100000000000000000000 (si si, los datos están mal estructurados porque son muy largos y confusos)

FK-2 Tipo_Habitacion a Hotel

- Acción: Insertar Tipo_Habitacion
- Datos caso exitoso: id_hotel=10
- Datos caso fallado: id_hotel= 100000000000000000000
- Esperado: Falla por FK, no existe un id_hotel 100000000000000000000 (si si, los datos están mal estructurados porque son muy largos y confusos)

FK-3 Habitacion a Hotel y a Tipo_Habitacion

- Acción: Insertar Habitacion
- Datos caso exitoso: id_hotel=10, id_tipo=1
- Dato 1 caso fallido: id_hotel=100000000000000000000
- Falla 2: id_tipo= 100000000000000000000
- Esperado: Falla por FK en cada caso, no existe un id_hotel ni id_tipo de ese estilo.

FK-4 Reserva a Cliente

- Acción: Insertar Reserva

- Dato caso exitoso: crear Cliente(usando PK osea el documento) y luego reserva con el identificador unico: documento_cliente="1111111111"

- Dato caso fallado: Reserva con documento_cliente=" 100000000000000000000"
- Esperado: Falla por FK, (probablemente no existen tantos cliente como para que uno tenga ese número)
-

FK-5 Reserva a Hotel y a Tipo_Habitacion

- Acción: Insertar Reserva

- Dato caso exitoso: id_hotel=10 existe, id_tipo=1

- Falla: id_hotel= 100000000000000000000 o id_tipo=100000000000000000000

- Esperado: Falla por FK, no existe un id_hotel o id_tipo con valor 100000000000000000000 (si si, los datos están mal estructurados porque son muy largos y confusos)

FK-6 Reserva_Servicio a Reserva y a Servicio

- Acción: Insertar Reserva_Servicio

- Dato caso exitoso: codigo_reserva="Reserva1" y id_servicio=5

- Dato 1 caso fallado: codigo_reserva="Reserva10000000000"

- Dato 2 caso fallado: id_servicio=100000000

- Esperado: Falla por FK, no existen estos datos porque son muy extensos por lo que los valores insertados se pueden considerar como no aptos.

FK-7 Tipo_habita_servicio a Tipo_Habitacion y a Servicio

- Acción: Insertar

- Dato caso exitoso: id_tipo=1 y id_servicio=5

- Dato caso fallado: id_tipo o id_servicio no existen
- Esperado: Falla por FK, si no existen ya de por si hacer falta requerimientos para poder guardar la información correctamente.

3) Pruebas de CHECK (restricciones)

CHECK-1 Tipo_Habitacion.capacidad mayor a 0

- Dato caso exitoso: capacidad=2
- Dato caso fallado: capacidad=0
- Esperado: Falla porque check tiene que ser un valor mayor a 0 para que sea valido

CHECK-2 Tipo_Habitacion.costo_alta mayor o igual a 0 y costo_baja mayor o igual a 0

- dato caso exitoso: costo_alta=200000, costo_baja=100000
- Caso dato fallado: costo_baja=-1000
- Esperado: Falla porque el costo tiene que ser valores positivos, algo no puede tener un valor menor que 0.

CHECK-3 Tipo_Habitacion.cantidad_disponible mayor o igual a 0

- dato caso exitoso: cantidad_disponible=10
- dato caso fallido: cantidad_disponible=-300
- Esperado: Falla porque el costo tiene que ser valores positivos, algo no puede tener un valor menor que 0.

CHECK-4 Servicio.tipo_costo está en "FIJO","CONSUMO"

- Dato caso exitoso: tipo_costo="FIJO"
- Dato caso fallado: tipo_costo="VARIABLE"
- Esperado: Falla por CHECK, no puede ser un dato variable

CHECK-5 Servicio.modo_cobro está en "POR RESERVA","POR NOCHE"

- dato caso exitoso: modo_cobro="POR NOCHE"
- dato caso fallido: modo_cobro="POR PERSONA"
- Esperado: Falla por CHECK, los cobros se hacen teniendo en cuenta las noches, no las personas por lo que el cobro no seria valido.

CHECK-6 Servicio.precio_cobro mayor o igual a 0

- dato caso exitoso: precio_cobro = 10000
- dato caso fallido: precio_cobro=-100
- Esperado: Falla por CHECK, como dicho antes, no pueden existir precios negativos.

CHECK-7 Reserva.num_adultos mayor o igual a 1 y num_menores mayor o igual a 0

- datos casos exitosos: num_adultos=1, num_menores=0
- dato 1 caso fallido: num_adultos=0
- dato 2 caso fallido : num_menores=-1
- Esperado: Falla por CHECK, no existe una reserva si no hay nadie responsable que pueda hacer la reserva y no puede existir un número de personas negativo.

CHECK-8 Reserva.estado está en "ACTIVA","CANCELADA","FINALIZADA"

- dato caso exitoso: estado="ACTIVA"

- dato caso fallido: estado="PENDIENTE"
- Esperado: Falla por CHECK, no pueden existir reservas pendientes, o está activa porque la hora de la reserva aún no es, o es cancelada si el cliente no quiso mantenerla o finalizada si ya la usaron, no hay más opciones.

CHECK-9 Reserva.precio_total mayor o igual a 0

- dato caso exitoso: precio_total=25000
- Falla: precio_total=-10
- Esperado: Falla por CHECK, no puede existir un precio de reserva negativo.

CHECK-10 Reserva_Servicio.cantidad mayor o igual a 1; subtotal mayor o igual a 0

- datos caso exitoso: cantidad=1, subtotal=25000
- dato caso fallido: cantidad=0
- Esperado: Falla por CHECK, no puede existir un servicio con valor 0 porque significa que no hay servicio por lo que no existiría.

CHECK-11 Tipo_habita_servicio.sobreCosto mayor o igual a 0 y habilitado esta en 0 y 1

- dato caso exitoso: sobreCosto=0, habilitado=1
- dato 1 caso fallido: sobreCosto=-1
- dato 2 caso fallido: habilitado=2
- Esperado: Falla por CHECK, no puede haber un sobre costco negativo, si es un sobre costo, es un adicional, por lo que siempre suma.

4) Pruebas de reglas de negocio

RN-1 No reservar si no hay disponibilidad

- Idea: si Tipo_Habitacion.cantidad_disponible=0, una reserva debería fallar ya que no se puede reservar si esta lleno

- Caso exitoso: cantidad_disponible=5 se va a reservar que se va a permitido para hacer la reservacion de manera exitosa

- Caso Fallido: cantidad_disponible=0 se va a reservar que se va a rechazado para dejarle saber al usuario que su petición es rechazada porque está lleno.

- Resultado esperado: el sistema no crea la reserva, solo las asigna.

RN-2 Fechas coherentes

- dats caso exitoso: fecha_entrada=2026-07-21, fecha_salida=2026-07-25

- datos caso fallido: fecha_salida=2026-07-09

- Esperado: el sistema rechaza la reserva ya que un usuario no puede reservar en una fecha y que ésta termine antes de empezar.

RN-3 Cancelación con plazo máximo de 3 días

- datos caso exitoso: cancelar 5 días antes que se va a permitido

- datos caso fallido: cancelar 1 día antes que se va a rechazado

- Esperado: el estado no debe cambiar a CANCELADA ya que no cumple con el requerimiento de aviso con antelación por lo que la reserva ya está establecida y es inmutable.

6. Conclusión

En conclusión, en esta entrega logramos transformar el problema planteado en un modelo de base de datos coherente y organizado, que representa de forma clara cómo funciona el sistema de reservas y cómo podemos usar modelos y tablas para resolver y probar el problema de forma clara y concisa. Cada parte importante del negocio quedó separada en su propia estructura, lo que permite mantener la información ordenada y evitar repeticiones innecesarias. Esto facilita que los datos sean consistentes y que las relaciones entre ellos tengan sentido.

El diseño no solo se enfoca en guardar información, sino también en asegurar su calidad. Las restricciones definidas permiten evitar errores comunes o innecesarios como valores negativos donde no se puede, valores astronómicamente grandes donde no hace sentido o datos duplicados. De esta manera, la base de datos no solo almacena información, sino que también ayuda a protegerla y mantenerla.

Las pruebas diseñadas demuestran que el modelo responde adecuadamente tanto en situaciones normales como en casos donde se intentan ingresar datos incorrectos, en los cuales se usan CHECK para que el sistema no se corra sin antes cumplir con los requisitos base. Esto permite validar que las reglas están bien implementadas y que el sistema puede manejar errores sin comprometer la integridad de la información. En general, el modelo cumple con los requerimientos planteados y deja una base sólida para el desarrollo de la aplicación.